

Retrieval-Augmented Generation (RAG)

Guide complet — de zéro à la production

Théorie, mathématiques, code Python et évaluation (récupération, génération, LLM-juge, RAGAS)

Édition fusionnée

Document de référence unifié : fondements mathématiques + pipeline pratique + évaluation complète + optimisation + techniques avancées.

Table des matières

Partie I — Le pipeline RAG	4
1 Pourquoi le RAG ?	4
2 Architecture générale d'un pipeline RAG	4
3 Fondements mathématiques des embeddings	5
3.1 Qu'est-ce qu'un embedding ?	5
3.2 Pourquoi des vecteurs denses plutôt que des mots-clés ?	5
4 Mesures de similarité	5
5 Recherche : du kNN exact à l'approché (ANN)	6
6 Embeddings symétriques vs asymétriques	6
6.1 Définitions	6
6.2 Pourquoi cette distinction est cruciale	7
6.3 Formulation mathématique de l'entraînement	7
6.4 Symétrie = cas particulier de l'asymétrie	7
6.5 Reconnaître un modèle asymétrique en pratique	7
6.6 Quand choisir l'un ou l'autre ?	8
7 Chunking	8
7.1 Les text splitters de LangChain	8
8 Le vector store : ChromaDB en pratique	10
8.1 API native : le module <code>chromadb</code>	10
8.2 Wrapper LangChain : <code>langchain_chroma</code>	10
9 Génération : formulation probabiliste	11
10 Code : le pipeline complet	12
Partie II — Optimisation du pipeline	13

11 Recherche hybride : dense + lexical	13
12 MMR : diversité des résultats	14
13 Reranking avec cross-encoders	14
14 Query transformation	14
15 Multi-vector retrieval : ColBERT	15
16 Mining de négatifs difficiles (hard negatives)	15
17 Filtrage par métadonnées	16
18 Contexte : compression & réordonnancement	16
Partie III — Évaluation d'un système RAG	16
19 Le triangle du RAG : quoi évaluer	16
20 Construire le jeu d'évaluation	16
21 Métriques de récupération (top- k)	17
21.1 Precision@ k , Recall@ k , F1@ k	17
21.2 Hit Rate@ k , MRR	17
21.3 MAP (Mean Average Precision)	17
21.4 NDCG@ k	17
21.5 Code : calculer ces métriques	18
22 Métriques de génération : classiques	18
23 Métriques de génération : LLM comme juge	19
24 RAGAS : évaluation automatisée	19
24.1 Faithfulness (fidélité) — arête $C \leftrightarrow a$	19
24.2 Answer Relevancy — arête $q \leftrightarrow a$	19
24.3 Context Precision — arête $q \leftrightarrow C$	20
24.4 Context Recall — arête $q \leftrightarrow C$ (avec référence)	20
24.5 Answer Correctness (avec référence)	20
24.6 Autres & limites	20
24.7 Code : RAGAS de bout en bout	21
25 Tableau de diagnostic	21
26 Méthodologie : la boucle fermée	21
Partie IV — Techniques RAG avancées	21
27 Self-RAG et Corrective RAG	21
28 Parent-Document Retrieval	22
29 GraphRAG	22
30 Agentic RAG	22
Récapitulatif et conclusion	22

31 La chaîne de décisions	22
32 Synthèse générale	22

Partie I — Le pipeline RAG

1 Pourquoi le RAG ?

Un grand modèle de langage (LLM) comme GPT-4, Claude ou Llama est entraîné sur un corpus *figé* à une date donnée. Cela produit trois limites fondamentales :

1. **Connaissance statique.** Le modèle ignore tout ce qui s'est passé après sa date de *knowledge cutoff*.
2. **Connaissance publique.** Il n'a jamais vu vos données privées : documents internes, emails, base de code, etc.
3. **Hallucinations.** En l'absence d'information, un LLM génère souvent une réponse plausible mais fausse, car son objectif d'entraînement est la vraisemblance textuelle, pas la véracité.

Définition 1 (RAG). *Le Retrieval-Augmented Generation est un paradigme qui, avant de répondre, va chercher (retrieve) des passages pertinents dans une base de connaissances externe, puis les injecte dans le prompt du LLM pour qu'il génère une réponse appuyée sur des sources.*

Formellement, si l'on note q la requête de l'utilisateur, C le corpus de documents, et p_θ la distribution du LLM paramétrée par θ , une génération *sans* RAG est simplement $y \sim p_\theta(\cdot \mid q)$. Avec RAG, on introduit un récupérateur R qui retourne un sous-ensemble $D_q \subset C$ de k passages les plus pertinents, et la génération devient :

$$y \sim p_\theta(\cdot \mid q, D_q), \quad D_q = R(q, C, k).$$

Tout l'art du RAG consiste à construire un R qui maximise la probabilité que D_q contienne effectivement l'information nécessaire à la réponse.

Idée principale

Bénéfices : **à jour** (on met à jour la base, pas le modèle), **traçable** (citations $\Rightarrow$ moins d'hallucinations), **privé**, et **moins cher** qu'un fine-tuning. Le RAG transforme un problème *linguistique* (comprendre le sens) en un problème *géométrique* (mesurer des distances entre vecteurs), que l'on sait traiter à grande échelle. Tout le pipeline découle de cette idée.

2 Architecture générale d'un pipeline RAG

Un pipeline RAG se décompose en deux phases distinctes.

Phase 1 — Indexation (hors ligne).

1. **Ingestion.** On charge les documents bruts (PDF, HTML, Markdown, bases SQL...).
2. **Nettoyage.** Suppression du boilerplate (menus, pieds de page), normalisation Unicode, détection de langue.
3. **Découpage (chunking).** Les documents sont segmentés en passages de taille contrôlée.
4. **Embedding.** Chaque chunk d_i est projeté dans un espace vectoriel dense : $e_i = f_{\text{enc}}(d_i) \in \mathbb{R}^d$.
5. **Indexation vectorielle.** Les vecteurs sont stockés dans une base vectorielle (FAISS, Milvus, Qdrant, pgvector, Weaviate, ChromaDB...) qui permet des recherches approximatives en temps sous-linéaire.

Phase 2 — Requête (en ligne).

1. **Embedding de la requête.** $e_q = f_{\text{enc}}^{(q)}(q)$.
2. **Recherche.** On récupère les k documents d_i maximisant un score de similarité $s(e_q, e_i)$.
3. **Augmentation du prompt.** Les chunks récupérés sont concaténés dans le contexte du LLM :

```
Contexte :  
[doc 1] [doc 2] ... [doc k]  
Question : <requete>  
Reponds uniquement en te basant sur le contexte ci-dessus.
```

4. **Génération.** Le LLM produit la réponse finale, idéalement avec citations.

La notation f_{enc} (pour les documents) et $f_{\text{enc}}^{(q)}$ (pour la requête) est volontairement distinguée : selon que ces deux fonctions sont identiques ou différentes, on parlera d'embedding **symétrique** ou **asymétrique** (section 6).

3 Fondements mathématiques des embeddings

3.1 Qu'est-ce qu'un embedding ?

Définition 2 (Fonction d'embedding). *Une embedding est une fonction $f : \mathcal{T} \rightarrow \mathbb{R}^d$ où $\mathcal{T}$ est l'ensemble des textes (séquences de tokens) et d la dimension du vecteur (typiquement 384, 768, 1024, 1536 ou 3072). Elle est apprise pour que la proximité géométrique dans $\mathbb{R}^d$ reflète la proximité sémantique dans $\mathcal{T}$: si $t_1 \approx_{\text{sens}} t_2$, alors $f(t_1) \approx f(t_2)$.*

Intuition

Espace **dense** ($d \sim$ centaines/milliers, coordonnées non nulles) : capture synonymes et contexte.
Espace **creux** (sac de mots) : capture les mots exacts. Le RAG moderne combine souvent les deux (recherche hybride, section 11).

3.2 Pourquoi des vecteurs denses plutôt que des mots-clés ?

Les approches lexicales classiques (TF-IDF, BM25) échouent dès qu'il y a *décalage de vocabulaire* : la requête « voiture électrique » ne matchera pas un document qui parle de « véhicule à batterie ». Les embeddings, eux, placent ces deux expressions dans la même région de $\mathbb{R}^d$.

4 Mesures de similarité

Soient $u, v \in \mathbb{R}^d$ (par exemple $u = E(q)$, $v = E(c)$).

Produit scalaire. $\langle u, v \rangle = \sum_{i=1}^d u_i v_i = u^\top v$. Encode direction *et* norme. Utile quand la norme a un sens (p.ex. confiance).

Similarité cosinus (référence en RAG).

$$\cos(u, v) = \frac{\langle u, v \rangle}{\|u\|_2 \|v\|_2} = \frac{\sum_i u_i v_i}{\sqrt{\sum_i u_i^2} \sqrt{\sum_i v_i^2}} \in [-1, 1].$$

Invariante par mise à l'échelle, elle ne mesure que l'angle entre vecteurs.

Distance euclidienne. $d_2(u, v) = \|u - v\|_2 = \sqrt{\sum_i (u_i - v_i)^2}$.

Proposition 1 (Équivalence sur la sphère unité). *Si les embeddings sont L_2 -normalisés ($\|u\|_2 = \|v\|_2 = 1$), alors*

$$\|u - v\|_2^2 = \|u\|^2 + \|v\|^2 - 2\langle u, v \rangle = 2 - 2\langle u, v \rangle = 2(1 - \cos(u, v)).$$

Démonstration. $\|u - v\|^2 = \|u\|^2 + \|v\|^2 - 2u^\top v = 1 + 1 - 2\cos\theta$. □

Idée principale

Sur la sphère unité, trier par distance euclidienne croissante ou par cosinus décroissant donne **exactement le même classement**. C'est pourquoi beaucoup de moteurs normalisent à l'ingestion puis utilisent le produit scalaire (plus rapide). On préfère le cosinus au produit scalaire brut pour qu'un texte long (grande norme) ne soit pas favorisé artificiellement.

5 Recherche : du kNN exact à l'approché (ANN)

Calculer $\cos(e_q, e_i)$ pour tous les N chunks coûte $O(Nd)$ et devient prohibitif dès $N \geq 10^6$. On utilise donc des structures **ANN** (Approximate Nearest Neighbours) :

- **HNSW** (Hierarchical Navigable Small World) : graphe en couches, recherche $O(\log N)$ en pratique, rappel très élevé.
- **IVF** (Inverted File) : partition de l'espace en n_{list} cellules de Voronoï via k-means, puis recherche seulement dans les $n_{\text{probe}} \ll n_{\text{list}}$ cellules les plus proches.
- **PQ** (Product Quantization) : compression des vecteurs par découpage en sous-vecteurs quantifiés, divisant la mémoire par 8 à $32\times$ au prix d'un léger coût de rappel.

Idée principale

Le compromis central est toujours : **rappel (recall) vs latence vs mémoire**. Le recall de l'index (fraction des vrais voisins retrouvés) est une *première* source d'erreur, en amont même des embeddings.

6 Embeddings symétriques vs asymétriques

C'est l'un des points les plus mal compris en RAG, et il détermine le choix du modèle d'embedding.

6.1 Définitions

Embedding symétrique. Un seul encodeur f encode aussi bien les requêtes que les documents :

$$e_q = f(q), \quad e_d = f(d), \quad s(q, d) = \cos(f(q), f(d)).$$

On suppose que q et d ont la même forme linguistique (même longueur, même style).

Embedding asymétrique. Deux encodeurs distincts, f_Q pour les requêtes et f_D pour les documents :

$$e_q = f_Q(q), \quad e_d = f_D(d), \quad s(q, d) = \cos(f_Q(q), f_D(d)).$$

Ou bien un unique modèle auquel on ajoute un *préfixe différent* pour les requêtes et les documents, ce qui revient effectivement à avoir deux fonctions distinctes.

6.2 Pourquoi cette distinction est cruciale

Dans un vrai moteur de recherche, une requête ressemble à « quel est le taux d'imposition des dividendes en France en 2024 ? » — 6 à 15 mots, formulation interrogative. Les documents du corpus sont des paragraphes de 200 à 500 mots, formulation déclarative. Un modèle symétrique supposerait à tort que ces deux entrées vivent dans la même distribution. Un modèle asymétrique apprend explicitement à rapprocher « courte question » et « long paragraphe qui y répond ».

6.3 Formulation mathématique de l'entraînement

L'entraînement repose sur l'**apprentissage contrastif**. On construit des triplets $(q_i, d_i^+, \{d_{i,j}^-\}_{j=1}^n)$ où q_i est une requête, d_i^+ un document pertinent (positif), et $d_{i,j}^-$ des documents non pertinents (négatifs). La perte la plus utilisée est **InfoNCE** (Noise-Contrastive Estimation), avec une température $\tau > 0$:

$$\mathcal{L}_{\text{InfoNCE}}(q_i) = -\log \frac{\exp(s(q_i, d_i^+)/\tau)}{\exp(s(q_i, d_i^+)/\tau) + \sum_{j=1}^n \exp(s(q_i, d_{i,j}^-)/\tau)}. \quad (1)$$

On reconnaît une cross-entropy sur une softmax : minimiser $\mathcal{L}_{\text{InfoNCE}}$ revient à maximiser la probabilité que d_i^+ soit le « bon » parmi $n + 1$ candidats.

Dérivation du gradient. Notons $z_j = s(q_i, d_j)/\tau$ et $p_j = \text{softmax}(z)_j = e^{z_j} / \sum_{\ell} e^{z_{\ell}}$. Alors

$$\frac{\partial \mathcal{L}_{\text{InfoNCE}}}{\partial z_j} = p_j - \mathbb{1}[j = +].$$

Le gradient pousse donc le score du positif vers le haut (par -1) et celui de tous les négatifs vers le bas (par $+p_j$), proportionnellement à leur probabilité actuelle. Les **hard negatives** — négatifs actuellement bien classés — ont un p_j élevé et fournissent donc l'essentiel du signal d'apprentissage. D'où l'importance de les miner (section 16).

6.4 Symétrie = cas particulier de l'asymétrie

Quand $f_Q = f_D = f$, on parle de **bi-encodeur partagé** (Siamese encoder). Sinon, c'est un bi-encodeur à poids distincts — parfois appelé **DPR** (Dense Passage Retrieval, Karpukhin et al. 2020) dans le cas historique.

6.5 Reconnaître un modèle asymétrique en pratique

Les modèles asymétriques modernes exposent souvent une instruction ou un préfixe différent :

```
1 # Modele symetrique (ex : all-MiniLM-L6-v2)
2 v_q = model.encode("Comment fonctionne le RAG ?")
3 v_d = model.encode("Le RAG combine recuperation et generation ...")
4
5 # Modele asymetrique (ex : BGE, E5, GTE-instruct)
6 v_q = model.encode("query: Comment fonctionne le RAG ?")
7 v_d = model.encode("passage: Le RAG combine recuperation ...")
```

Idée principale

Oublier le préfixe sur un modèle asymétrique dégrade le rappel de 10 à 30 %. C'est l'erreur débutant la plus fréquente.

6.6 Quand choisir l'un ou l'autre ?

Cas d'usage	Type	Exemples
Question-Réponse, recherche web (courte requête → long doc)	Asymétrique	BGE-M3, E5-large, text-embedding-3
Détection de doublons, clustering de phrases	Symétrique	all-MiniLM, SBERT
Similarité sémantique pure (STS)	Symétrique	SBERT-sts
Recherche de code à partir de doc-string	Asymétrique	code-search

7 Chunking

Découper en segments de longueur $\approx L$ avec recouvrement o : on obtient $\approx |D|/(L - o)$ chunks.

Idée principale

Le chunk est l'**unité atomique de récupération**. L grand $\Rightarrow$ embedding qui « moyenne » plusieurs idées (recherche floue) ; L petit $\Rightarrow$ contexte tronqué. Le recouvrement o évite de couper une idée à la frontière. C'est le levier le plus sous-estimé : un mauvais découpage = les bonnes informations ne sont jamais ensemble dans un même chunk. On règle L, o empiriquement, via le *context recall*.

Familles de découpage.

- **Chunking fixe.** Fenêtre glissante de T tokens avec recouvrement o . Simple mais coupe au milieu des phrases.
- **Chunking récursif.** On coupe d'abord sur `\n\n`, sinon `\n`, sinon `.`, sinon l'espace, sinon le caractère — en respectant une taille cible. C'est le défaut de `LangChain`.
- **Chunking sémantique.** On embedding chaque phrase $p_1, \dots, p_n$, puis on coupe entre p_i et p_{i+1} quand $1 - \cos(f(p_i), f(p_{i+1})) > \theta$ (seuil souvent au 95^e percentile des distances). On respecte les frontières thématiques.
- **Late chunking.** Technique récente (2024) : embedding tout le document en une passe (modèle à long contexte), puis moyenne des tokens par chunk. Conserve le contexte global tout en gardant la granularité chunk.

Choix de T . T petit (100–200 tokens) : haute précision, mais risque de perdre le contexte. T grand (800–1500 tokens) : plus de contexte, mais dilue le signal et coûte plus de tokens à la génération. **En pratique : $T \approx 256$ –512 tokens, recouvrement $o \approx 0.1 T$.**

7.1 Les text splitters de LangChain

CharacterTextSplitter. Coupe sur un seul séparateur, puis regroupe jusqu'à `chunk_size`.

```

1 from langchain.text_splitter import CharacterTextSplitter
2 splitter = CharacterTextSplitter(
3     separator="\n\n", chunk_size=800, chunk_overlap=100)
4 chunks = splitter.split_text(texte) # liste de str
5 docs = splitter.create_documents([texte]) # liste de Document

```

Simple mais rigide : si un paragraphe dépasse `chunk_size`, il n'est pas recoupé finement. À réserver aux textes bien structurés.

RecursiveCharacterTextSplitter (le défaut). Essaie une hiérarchie de séparateurs récursivement : paragraphes, puis lignes, puis mots, puis caractères.

```
1 from langchain.text_splitter import RecursiveCharacterTextSplitter
2 splitter = RecursiveCharacterTextSplitter(
3     chunk_size=800, chunk_overlap=100,
4     separators=["\n\n", "\n", " ", ""], # ordre de priorite
5     length_function=len) # ou un compteur de tokens
6 chunks = splitter.split_documents(docs)
```

Idée principale

Respecter la structure naturelle du texte le plus longtemps possible : couper d'abord aux frontières « fortes » (paragraphes) et seulement ensuite aux frontières « faibles » garde des unités sémantiquement cohérentes. C'est le splitter recommandé par défaut.

TokenTextSplitter. Découpe en comptant les tokens (via tiktoken).

```
1 from langchain.text_splitter import TokenTextSplitter
2 splitter = TokenTextSplitter(chunk_size=256, chunk_overlap=32)
3 chunks = splitter.split_text(texte)
```

Pourquoi ça aide

Les LLM et modèles d'embedding ont une limite en *tokens*, pas en caractères. Compter en tokens garantit qu'aucun chunk ne dépasse la fenêtre $\Rightarrow$ pas de troncature silencieuse.

Splitters structurés : code et Markdown.

```
1 from langchain.text_splitter import (RecursiveCharacterTextSplitter,
2     Language, MarkdownHeaderTextSplitter)
3 py = RecursiveCharacterTextSplitter.from_language(
4     language=Language.PYTHON, chunk_size=500, chunk_overlap=0)
5 md = MarkdownHeaderTextSplitter(headers_to_split_on=[
6     ("#", "h1"), ("##", "h2"), ("###", "h3")])
7 sections = md.split_text(markdown_text) # chaque chunk porte ses titres
```

Pourquoi ça aide

Découper selon la structure (fonction, section) plutôt qu'une longueur arbitraire évite de couper au milieu d'une idée ; les métadonnées de titre permettent un filtrage fin à la recherche.

SemanticChunker. Coupe là où le sens change : (1) découper en phrases ; (2) encoder chaque phrase ; (3) distance cosinus entre phrases consécutives ; (4) coupure quand la distance dépasse un seuil (p.ex. 95^e percentile) ; (5) regrouper les phrases entre deux coupures.

```
1 from langchain_experimental.text_splitter import SemanticChunker
2 from langchain_huggingface import HuggingFaceEmbeddings
3 emb = HuggingFaceEmbeddings(
4     model_name="sentence-transformers/all-MiniLM-L6-v2")
5 splitter = SemanticChunker(emb, breakpoint_threshold_type="percentile")
6 chunks = splitter.create_documents([texte])
```

Idée principale

Les frontières de chunk doivent tomber aux *ruptures de sujet*, pas à un nombre fixe de caractères. Résultat : des chunks très cohérents (embeddings plus « purs », $\uparrow$ context precision), au prix d'un coût de calcul plus élevé (encoder toutes les phrases).

Comment choisir.

Splitter	Quand	Coût
Recursive (caractères)	défaut, texte général	faible
Token	respecter la limite du modèle	faible
from_language / Markdown	code, docs structurés	faible
Semantic	cohérence maximale des chunks	élevé (embeddings)

8 Le vector store : ChromaDB en pratique

Chroma stocke les vecteurs et fait la recherche des plus proches voisins. Deux niveaux d'API : le module `chromadb` (client natif, contrôle fin) et le wrapper `langchain_chroma` (intégré au pipeline LangChain).

8.1 API native : le module chromadb

```
1 import chromadb
2 # Persistant : les données survivent au redémarrage
3 client = chromadb.PersistentClient(path="./chroma_db")
4 # (ou chromadb.Client() pour un client éphémère en mémoire)
5
6 # Une "collection" = une table de vecteurs. hnsw:space fixe la distance.
7 col = client.get_or_create_collection(
8     name="rag_docs", metadata={"hnsw:space": "cosine"}) # cosine / l2 / ip
9
10 col.add(
11     ids=["d1", "d2", "d3"],
12     documents=["Paris est la capitale.", "Lyon est une ville.",
13               "La Seine traverse Paris."],
14     metadatas=[{"source": "geo.pdf", "page": 1},
15                {"source": "geo.pdf", "page": 2},
16                {"source": "geo.pdf", "page": 3}],
17     # embeddings=[...], [...], [...]] # optionnel si fournis à la main
18 )
19 col.upsert(ids=["d1"], documents=["Paris, capitale de la France."]) # ajoute ou
20 met à jour
21
22 # Recherche top-k, avec filtre sur métadonnées (where) et sur texte
23 (where_document)
24 res = col.query(
25     query_texts=["Quelle est la capitale ?"], n_results=2,
26     where={"source": "geo.pdf"},
27     where_document={"$contains": "capitale"})
28 print(res["documents"], res["distances"])
29
30 col.get(ids=["d1"]) # récupérer par id
31 col.get(where={"page": {"$gte": 2}}) # $eq $ne $gt $gte $lt $in $and $or
32 col.update(ids=["d2"], metadatas=[{"source": "geo_v2.pdf"}])
33 col.delete(ids=["d3"]) # ou delete(where={...})
34 print(col.count()) # nombre de vecteurs
```

8.2 Wrapper LangChain : langchain_chroma

Plus pratique pour le RAG : il branche directement un modèle d'embedding et expose un *retriever*.

```
1 from langchain_chroma import Chroma
2 from langchain_huggingface import HuggingFaceEmbeddings
3 emb = HuggingFaceEmbeddings()
```

```

4     model_name="sentence-transformers/all-MiniLM-L6-v2")
5
6 # (a) construire depuis des documents (encode + stocke)
7 vs = Chroma.from_documents(
8     documents=chunks, embedding=emb,
9     collection_name="rag_docs", persist_directory="./chroma_db",
10    collection_metadata={"hnsw:space": "cosine"})
11
12 # (b) construire depuis des textes bruts
13 vs = Chroma.from_texts(texts=["...", "..."], embedding=emb,
14    metadatas=[{"source": "a"}, {"source": "b"}], ids=["t1", "t2"])
15
16 # (c) RECHARGER une base existante (sans re-encoder)
17 vs = Chroma(collection_name="rag_docs", embedding_function=emb,
18    persist_directory="./chroma_db")

```

```

1 vs.add_documents(documents=nouveaux_chunks, ids=["x1", "x2"])
2 docs = vs.similarity_search("capitale de la France", k=4)
3 pairs = vs.similarity_search_with_score("capitale", k=4) # plus petit = plus
    proche
4 docs = vs.similarity_search("ville", k=4, filter={"source": "geo.pdf"})
5 docs = vs.similarity_search_by_vector(emb.embed_query("Paris"), k=4)
6 # Diversité (MMR) : recupere fetch_k puis garde k diversifies
7 docs = vs.max_marginal_relevance_search("Paris", k=4, fetch_k=20,
    lambda_mult=0.5)
8 vs.delete(ids=["x1"])

```

```

1 # Transformer en retriever pour la chaîne RAG
2 retriever = vs.as_retriever(search_kwargs={"k": 4}) #
    similarite simple
3 retriever = vs.as_retriever(search_type="mmr", #
    diversite
4     search_kwargs={"k": 4, "fetch_k": 20, "lambda_mult": 0.5})
5 retriever = vs.as_retriever(search_type="similarity_score_threshold", # seuil
6     search_kwargs={"score_threshold": 0.5, "k": 4})
7 retriever = vs.as_retriever(search_kwargs={"k": 4, "filter": {"source":
    "geo.pdf"}})

```

Idée principale

Le **retriever** est le pont entre le vector store et la chaîne RAG : c'est l'objet qu'on branche dans le pipeline (`retriever | prompt | llm`). Ses options (`k`, `mmr`, `filter`, `score_threshold`) sont autant de leviers d'optimisation de la récupération, mesurables via *context precision/recall*. On utilise `chromadb` natif pour un contrôle fin et `langchain_chroma` pour brancher la base dans une chaîne. Fixez la distance à `cosine` et normalisez vos embeddings.

9 Génération : formulation probabiliste

Le LLM génère token par token, conditionné sur question *et* contexte :

$$P_{\theta}(y \mid q, c) = \prod_{t=1}^T P_{\theta}(y_t \mid y_{<t}, q, c).$$

Comme le retrieveur renvoie k documents, le modèle RAG *marginalise* sur le document latent c , pondéré par la probabilité de récupération $P_\eta(c | q) = \text{softmax}(s(q, c)/\tau)$:

$$\text{RAG-Sequence : } P(y | q) \approx \sum_{c \in \text{top-}k(q)} P_\eta(c | q) P_\theta(y | q, c),$$

$$\text{RAG-Token : } P(y | q) \approx \prod_t \sum_{c \in \text{top-}k(q)} P_\eta(c | q) P_\theta(y_t | y_{<t}, q, c).$$

Idée principale

La réponse est conditionnée à une **moyenne pondérée** des k documents, chaque poids étant la confiance de récupération. En pratique on concatène les chunks dans le prompt : une approximation où le LLM pondère implicitement.

10 Code : le pipeline complet

Stack locale : langchain (orchestration), chromadb (base vectorielle), sentence-transformers (embeddings), ollama (LLM local, Llama 3.1).

```
1 pip install langchain langchain-community langchain-chroma \
2   langchain-huggingface langchain-ollama \
3   chromadb sentence-transformers pypdf
4 # ollama pull llama3.1

1 # Indexation : charger -> decouper -> encoder -> stocker
2 from langchain_community.document_loaders import PyPDFLoader
3 from langchain.text_splitter import RecursiveCharacterTextSplitter
4 from langchain_huggingface import HuggingFaceEmbeddings
5 from langchain_chroma import Chroma
6
7 docs = PyPDFLoader("mon_document.pdf").load()
8 chunks = RecursiveCharacterTextSplitter(
9     chunk_size=800, chunk_overlap=100).split_documents(docs)
10 emb = HuggingFaceEmbeddings(
11     model_name="sentence-transformers/all-MiniLM-L6-v2")
12 vs = Chroma.from_documents(chunks, emb, persist_directory="./chroma_db")

1 # Recuperation + generation (LCEL)
2 from langchain_ollama import ChatOllama
3 from langchain_core.prompts import ChatPromptTemplate
4 from langchain_core.runnables import RunnablePassthrough
5 from langchain_core.output_parsers import StrOutputParser
6
7 retriever = vs.as_retriever(search_kwargs={"k": 4})
8 llm = ChatOllama(model="llama3.1", temperature=0)
9 prompt = ChatPromptTemplate.from_template(
10     "Reponds UNIQUEMENT a partir du contexte. Si absent, dis-le.\n"
11     "Contexte :\n{context}\n\nQuestion : {question}\nReponse :")
12
13 def fmt(ds): return "\n\n".join(d.page_content for d in ds)
14
15 chain = ({ "context": retriever | fmt, "question": RunnablePassthrough()
16     | prompt | llm | StrOutputParser()
17 print(chain.invoke("Quelle est la conclusion du document ?"))
```

Partie II — Optimisation du pipeline

Un RAG naïf (charger des PDF, chunker à 500 tokens, embedder, chercher top-5, injecter dans le prompt) donne rarement plus de 60 % de réponses correctes sur un benchmark métier. Voici les leviers, du plus simple au plus avancé. *Règle d'or* : on optimise **un levier à la fois**, puis on re-mesure (section 26).

11 Recherche hybride : dense + lexical

Les embeddings échouent sur : noms propres rares, identifiants produit, codes (« ISO 27001 »), termes métier non vus à l'entraînement. Le **BM25** (lexical) excelle là où les embeddings échouent, et réciproquement. On les combine.

Rappel du BM25. Pour une requête $q = (t_1, \dots, t_m)$ et un document d :

$$\text{BM25}(q, d) = \sum_{i=1}^m \text{IDF}(t_i) \cdot \frac{f(t_i, d) (k_1 + 1)}{f(t_i, d) + k_1 (1 - b + b \frac{|d|}{\text{avgdl}})},$$

où $f(t_i, d)$ est la fréquence du terme t_i dans d , $|d|$ la longueur du document, avgdl la longueur moyenne, et

$$\text{IDF}(t_i) = \log \frac{N - n(t_i) + 0.5}{n(t_i) + 0.5},$$

avec N le nombre total de documents et $n(t_i)$ le nombre contenant t_i . Paramètres usuels : $k_1 \in [1.2, 2.0]$ et $b = 0.75$.

Intuition

k_1 contrôle la *saturation* de la fréquence (au-delà d'un certain nombre d'occurrences, un terme n'apporte plus de gain) ; b contrôle la *normalisation par la longueur* (à $b = 0$ on ignore la longueur, à $b = 1$ on la prend pleinement en compte). L'IDF pénalise les termes très fréquents (peu discriminants) et favorise les termes rares.

Fusion. Deux approches.

1. Combinaison linéaire après normalisation :

$$s_{\text{hybride}}(q, d) = \alpha \tilde{s}_{\text{dense}}(q, d) + (1 - \alpha) \tilde{s}_{\text{BM25}}(q, d), \quad \alpha \in [0, 1],$$

α étant un hyperparamètre à calibrer (souvent $\alpha \approx 0.6$ – 0.7). La normalisation ($\tilde{s}$) est nécessaire car les deux scores ne vivent pas sur la même échelle.

2. Reciprocal Rank Fusion (RRF). Ne nécessite pas de normalisation, seulement des rangs :

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{\kappa + r(d)}, \quad \kappa = 60 \text{ par convention},$$

où $r(d)$ est le rang du document d dans la liste r , et $\mathcal{R}$ l'ensemble des listes à fusionner (BM25, dense, voire plusieurs encodeurs). Simple, robuste, étonnamment efficace.

```
1 # Hybride BM25 + dense via LangChain
2 from langchain_community.retrievers import BM25Retriever
3 from langchain.retrievers import EnsembleRetriever
4 bm25 = BM25Retriever.from_documents(chunks); bm25.k = 4
5 hybrid = EnsembleRetriever(
6     retrievers=[bm25, vs.as_retriever(search_kwargs={"k": 4})],
7     weights=[0.4, 0.6])
```

Pourquoi ça aide

Dense = sens, sparse = termes exacts ; fusion par rangs $\Rightarrow \uparrow$ context recall robuste. C'est aujourd'hui l'ossature standard d'un RAG sérieux.

12 MMR : diversité des résultats

La recherche par similarité pure renvoie souvent k chunks quasi-redondants. Le **Maximal Marginal Relevance** pénalise la redondance :

$$\text{MMR} = \arg \max_{c \notin S} \left[\lambda \text{sim}(q, c) - (1 - \lambda) \max_{c' \in S} \text{sim}(c, c') \right],$$

où S est l'ensemble déjà sélectionné et λ règle le compromis pertinence/diversité.

Pourquoi ça aide

Pénaliser la redondance $\Rightarrow$ plus d'information *distincte* dans le contexte $\Rightarrow \uparrow$ context recall. Voir `max_marginal_relevance_search` (section ChromaDB).

13 Reranking avec cross-encoders

Un **bi-encodeur** calcule $s(q, d) = \cos(f(q), f(d))$: l'interaction entre q et d se fait uniquement via leurs vecteurs finaux. C'est rapide (on pré-calcule les $f(d)$) mais peu expressif.

Un **cross-encoder** prend $[q, d]$ concaténé en entrée d'un transformer et prédit directement un score :

$$s_{\text{CE}}(q, d) = \text{MLP}_{\text{BERT}}([\text{CLS}] q [\text{SEP}] d [\text{SEP}]) \in \mathbb{R}.$$

Les tokens de q et de d s'attendent mutuellement dans les couches d'attention. Beaucoup plus précis, mais le score ne peut pas être pré-calculé : il faut un forward pass par couple (q, d) .

Pipeline en deux étapes (retrieve-then-rerank).

1. Le bi-encodeur récupère K candidats (p.ex. $K = 50$ ou 100).
2. Le cross-encoder rescore ces K candidats ; on garde les k meilleurs ($k = 5$ p.ex.).

Gain typique : +10 à +20 points de nDCG à coût négligeable. Exemples : `bge-reranker-v2`, Cohere Rerank 3, `ms-marco-MiniLM`.

```
1 from langchain.retrievers import ContextualCompressionRetriever
2 from langchain.retrievers.document_compressors import CrossEncoderReranker
3 from langchain_community.cross_encoders import HuggingFaceCrossEncoder
4 ce = HuggingFaceCrossEncoder(model_name="cross-encoder/ms-marco-MiniLM-L-6-v2")
5 rr = ContextualCompressionRetriever(
6     base_compressor=CrossEncoderReranker(model=ce, top_n=4),
7     base_retriever=vs.as_retriever(search_kwargs={"k": 50}))
```

Pourquoi ça aide

Le cross-encoder lit la paire (q, c) *ensemble* (précis) $\Rightarrow \uparrow$ context precision/NDCG $\Rightarrow$ moins de bruit dans le prompt $\Rightarrow \uparrow$ faithfulness.

14 Query transformation

La requête brute est souvent sous-optimale : ambiguë, trop courte, mal formulée. On la transforme avant la recherche.

HyDE (Hypothetical Document Embeddings). Au lieu d’embedder la question, on demande d’abord au LLM de générer une *réponse hypothétique* $\tilde{d}$, puis on embedde cette réponse :

$$e_q = f_D(\tilde{d}), \quad \tilde{d} \sim p_\theta(\cdot | q).$$

Justification : $\tilde{d}$ vit dans la même distribution textuelle que les documents du corpus (même longueur, même style déclaratif), donc la recherche vecteur-à-vecteur est mieux conditionnée. Gain : +5 à +15 % de rappel, surtout utile avec des modèles symétriques.

Multi-query. On demande au LLM de reformuler la requête en m variantes $(q_1, \dots, q_m)$, on lance m recherches, et on fusionne via RRF. Utile pour les requêtes ambiguës.

```
1 from langchain.retrievers.multi_query import MultiQueryRetriever
2 mq = MultiQueryRetriever.from_llm(retriever=vs.as_retriever(), llm=llm)
```

Décomposition. Pour une question complexe (« Quelle entreprise a le plus gros bénéfice parmi celles qui ont un ratio dette/capitaux propres < 0,5 ? »), on la décompose en sous-questions, on résout chacune, puis on recombine.

Pourquoi ça aide

HyDE comble le fossé « style question / style document » $\Rightarrow \uparrow$ recall sur requêtes difficiles ; multi-query couvre plusieurs formulations ; la décomposition débloque le raisonnement multi-sauts.

15 Multi-vector retrieval : ColBERT

L’approche bi-encodeur compresse un document entier en *un seul* vecteur — beaucoup d’information est perdue. **ColBERT** (Khattab & Zaharia, 2020) propose une *interaction tardive* (late interaction) : chaque token du document et de la requête a son propre vecteur. Pour une requête de n tokens et un document de m tokens :

$$s_{\text{ColBERT}}(q, d) = \sum_{i=1}^n \max_{j=1, \dots, m} \langle e_{q_i}, e_{d_j} \rangle.$$

C’est le **MaxSim** : chaque token de la requête trouve le token du document qui lui correspond le mieux, et on somme. On retient la granularité token-niveau tout en gardant la possibilité de pré-calculer les embeddings du document. Coût mémoire : $\sim 100\times$ celui du bi-encodeur, mais compressible par quantification (ColBERTv2 avec PLAID).

16 Mining de négatifs difficiles (hard negatives)

Comme vu avec le gradient de l’InfoNCE (équation 1), les **hard negatives** sont essentiels à la qualité d’un modèle d’embedding. Plusieurs stratégies :

- **In-batch negatives.** Dans un batch de B paires (q_i, d_i^+) , on utilise les d_j^+ des autres paires ($j \neq i$) comme négatifs. Gratuit mais souvent trop faciles.
- **BM25 hard negatives.** On prend des documents bien classés par BM25 mais qui ne sont pas les vrais positifs. Force le modèle dense à dépasser le lexical.
- **Self-mined negatives.** À chaque époque, on utilise le modèle en cours pour retrouver les top- k faux positifs et les ajouter comme négatifs de l’époque suivante.

17 Filtrage par métadonnées

On attache à chaque chunk des métadonnées : `source`, `date`, `auteur`, `type_document`, `departement`. La recherche devient

$$\arg \max_{d \in C : \phi(d) = \text{vrai}} s(q, d),$$

où ϕ est un prédicat filtrant (ex. : `date ≥ 2023 ET departement = RH`). On parle de **pre-filtering** (filtrer avant la recherche ANN) ou **post-filtering**. Le pré-filtrage est plus précis mais peut casser l'index ANN ; les bases modernes (Qdrant, Milvus, Chroma) gèrent cela proprement.

18 Contexte : compression & réordonnancement

Pourquoi ça aide

Contre le « lost in the middle » (les LLM exploitent mal l'information située au *milieu* d'un long contexte) : on *compresse* le contexte (retirer le bruit) et on *réordonne* pour placer le plus pertinent aux extrémités $\Rightarrow \uparrow$ faithfulness.

Partie III — Évaluation d'un système RAG

Sans évaluation, on optimise à l'aveugle. Un RAG a deux étages à évaluer *séparément* : la récupération et la génération.

19 Le triangle du RAG : quoi évaluer

On évalue les trois arêtes du triangle (question q , contexte C , réponse a) :

Arête	Question	Maillon
$q \leftrightarrow C$	Contexte pertinent ?	Récupération
$C \leftrightarrow a$	Réponse fondée sur le contexte ?	Génération (fidélité)
$q \leftrightarrow a$	Réponse répond à la question ?	Génération (pertinence)

Idée principale

Évaluer = **localiser la panne**. Mauvaise réponse = mauvais contexte (arête $q-C$) ou mauvaise exploitation d'un bon contexte (arête $C-a$) ? Chaque famille de métriques répond à une arête et oriente l'optimisation.

20 Construire le jeu d'évaluation

Un échantillon contient : q (question), a (réponse générée), C (contextes récupérés), g (référence / ground truth). Construction **manuelle** (fiable, coûteuse) ou **synthétique** (un LLM génère les questions depuis les chunks). Distinction clé :

- métriques **avec référence** : context recall, answer correctness ;
- métriques **sans référence** : faithfulness, answer relevancy, context precision — utilisables en production.

21 Métriques de récupération (top- k)

On note R_q l'ensemble des documents pertinents (vérité terrain) et $\text{rel}(i) = 1$ si le document au rang i est pertinent.

Exemple fil rouge. 5 documents récupérés, $|R_q| = 4$, pertinences sur la liste : (1, 0, 1, 0, 1).

21.1 Precision@ k , Recall@ k , F1@ k

$$P@k = \frac{\text{\#pertinents dans top-}k}{k}, \quad R@k = \frac{\text{\#pertinents dans top-}k}{|R_q|}, \quad F1@k = \frac{2 P@k R@k}{P@k + R@k}.$$

k	cumul pertinents	$P@k$	$R@k$ ($ R_q = 4$)
1	1	1,00	0,25
3	2	0,67	0,50
5	3	0,60	0,75

$$F1@5 = \frac{2 \cdot 0,60 \cdot 0,75}{0,60 + 0,75} = \frac{0,90}{1,35} \approx 0,667.$$

Intuition

$P@k$: part *utile* de ce qu'on sort ; $R@k$: part de l'utile *retrouvée*. Quand k croît, R monte, P baisse : compromis précision/rappel.

21.2 Hit Rate@ k , MRR

$$\text{HitRate}@k = \frac{1}{|Q|} \sum_q \mathbb{1}[\exists i \leq k : \text{rel}(i) = 1], \quad \text{MRR} = \frac{1}{|Q|} \sum_q \frac{1}{\text{rang du 1}^{\text{er}} \text{ pertinent}}.$$

Exemple : premier pertinent au rang 1 $\Rightarrow \text{RR} = 1$. Avec une 2^e requête (1^{er} pertinent au rang 3) : $\text{MRR} = \frac{1}{2} \left(1 + \frac{1}{3}\right) = \frac{2}{3}$.

21.3 MAP (Mean Average Precision)

$$\text{AP} = \frac{1}{|R_q|} \sum_k P@k \cdot \text{rel}(k), \quad \text{MAP} = \frac{1}{|Q|} \sum_q \text{AP}_q.$$

Pertinents aux rangs 1, 3, 5, $|R_q| = 4$:

$$\text{AP} = \frac{1}{4} \left(\underbrace{1}_{P@1} + \underbrace{0,667}_{P@3} + \underbrace{0,600}_{P@5} \right) = \frac{2,267}{4} \approx 0,567.$$

Le pertinent manqué contribue 0 : la MAP pénalise rangs *et* oublis.

21.4 NDCG@ k

$$\text{DCG}@k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)} \quad \left(\text{binaire : } \sum \frac{\text{rel}_i}{\log_2(i+1)} \right), \quad \text{NDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k},$$

où $\text{IDCG}@k$ est le DCG du classement idéal.

$$\text{DCG}@5 = \frac{1}{\log_2 2} + \frac{1}{\log_2 4} + \frac{1}{\log_2 6} = 1 + 0,5 + 0,387 = 1,887.$$

Ordre idéal (3 pertinents en tête) : $\text{IDCG}@5 = 1 + 0,631 + 0,5 = 2,131$, donc $\text{NDCG}@5 = 1,887/2,131 \approx 0,885$.

Idée principale

Le facteur $1/\log_2(i+1)$ encode « un bon doc au rang 1 vaut plus qu'au rang 10 » ; la normalisation par l'IDCG ramène dans $[0, 1]$. La version graduée gère des pertinences « un peu / beaucoup / parfait ».

21.5 Code : calculer ces métriques

```
1 import numpy as np
2
3 def precision_at_k(rel, k): return sum(rel[:k]) / k
4 def recall_at_k(rel, k, R): return sum(rel[:k]) / R
5
6 def reciprocal_rank(rel):
7     for i, r in enumerate(rel, 1):
8         if r: return 1.0 / i
9     return 0.0
10
11 def average_precision(rel, R):
12     score, hits = 0.0, 0
13     for i, r in enumerate(rel, 1):
14         if r:
15             hits += 1
16             score += hits / i      # = Precision@i quand rel(i)=1
17     return score / R
18
19 def dcg_at_k(rel, k):
20     return sum(r / np.log2(i + 1) for i, r in enumerate(rel[:k], 1))
21
22 def ndcg_at_k(rel, k):
23     ideal = dcg_at_k(sorted(rel, reverse=True), k)
24     return dcg_at_k(rel, k) / ideal if ideal else 0.0
25
26 rel = [1, 0, 1, 0, 1]; R = 4
27 print(precision_at_k(rel, 5), recall_at_k(rel, 5, R)) # 0.6 0.75
28 print(average_precision(rel, R)) # ~0.567
29 print(ndcg_at_k(rel, 5)) # ~0.885
```

22 Métriques de génération : classiques

Comparaison de la réponse a à une référence g par chevauchement de texte.

- **ROUGE-L** (plus longue sous-séquence commune, LCS) : $R = \frac{LCS}{|g|}$, $P = \frac{LCS}{|a|}$, $F = \frac{(1+\beta^2)PR}{R+\beta^2P}$.
- **BLEU** : $BLEU = BP \cdot \exp(\sum_n w_n \log p_n)$, p_n précision n-grammes, BP pénalité de brièveté.
- **BERTScore** : appariement glouton entre embeddings de tokens, puis P/R/F sur les cosinus.
- **Token-F1 / Exact Match** (style SQuAD) : recouvrement de tokens / égalité exacte.

Intuition

Rapides et reproductibles, mais **myopes** : elles récompensent les mots communs, pas la vérité. Une bonne réponse *reformulée* a un ROUGE faible ; une fausse réponse qui copie le vocabulaire a un ROUGE élevé. D'où le passage aux méthodes à base de LLM.

```
1 from rouge_score import rouge_scorer
2 from sacrebleu import sentence_bleu
3 from bert_score import score as bertscore
```

```

4
5 ref = "Paris est la capitale de la France."
6 hyp = "La capitale de la France est Paris."
7
8 rl = rouge_scorer.RougeScorer(["rougeL"], use_stemmer=True)
9 print(rl.score(ref, hyp)["rougeL"].fmeasure) # F-mesure ROUGE-L
10 print(sentence_bleu(hyp, [ref]).score) # BLEU
11 P, R, F = bertscore([hyp], [ref], lang="fr")
12 print(F.item()) # F BERTScore

```

23 Métriques de génération : LLM comme juge

Un LLM puissant note la réponse selon une consigne : *note directe* (échelle 1–5 avec grille) ou *comparaison par paires* (quel système est meilleur ?).

Pourquoi ça aide

Biais : position (favorise la 1^{re} réponse), verbosité (favorise le long), auto-préférence. **Parades** : permuter l'ordre et moyenner, fournir une grille explicite, demander un raisonnement *avant* la note, calibrer sur quelques exemples humains.

```

1 JUDGE = """Tu es un evaluateur. Note de 1 a 5 la REPONSE selon :
2 - exactitude vis-a-vis du CONTEXTE
3 - pertinence vis-a-vis de la QUESTION
4 Reponds au format JSON : {"note": <1-5>, "raison": "<...>"}.
5
6 QUESTION : {q}
7 CONTEXTE : {ctx}
8 REPONSE : {a}"""
9
10 def llm_judge(llm, q, ctx, a):
11     out = llm.invoke(JUDGE.format(q=q, ctx=ctx, a=a))
12     return out.content # parser le JSON ensuite

```

24 RAGAS : évaluation automatisée

RAGAS automatise l'évaluation de bout en bout via un LLM-juge et des embeddings, souvent *sans* référence. Principe : décomposer en unités atomiques, vérifier/comparer, puis agréger dans $[0, 1]$.

24.1 Faithfulness (fidélité) — arête $C \leftrightarrow a$

Le LLM décompose a en affirmations $\{s_1, \dots, s_n\}$, verdict $v_i \in \{0, 1\}$ (inférable du contexte) :

$$\text{Faithfulness} = \frac{1}{n} \sum_{i=1}^n v_i.$$

Exemple : $a = \text{« Paris est la capitale et compte 10 millions d'habitants »}$; contexte = « capitale... 2,1 millions ». $s_1 \Rightarrow v_1 = 1$, $s_2 \Rightarrow v_2 = 0 \Rightarrow \text{Faithfulness} = 1/2 = 0,5$.

24.2 Answer Relevancy — arête $q \leftrightarrow a$

Le LLM génère n questions $\{q'_i\}$ depuis la réponse, puis :

$$\text{AnswerRelevancy} = \frac{1}{n} \sum_{i=1}^n \cos(E(q), E(q'_i)).$$

Idée principale

Si la réponse est précise, les questions qu'elle « inspire » ressemblent à q (cosinus élevé) ; vague/hors-sujet $\Rightarrow$ cosinus faible. Mesuré *sans* référence.

24.3 Context Precision — arête $q \leftrightarrow C$

Pertinence $v_k \in \{0, 1\}$ par chunk ; AP sur la liste :

$$\text{ContextPrecision@K} = \frac{\sum_{k=1}^K (P@k \cdot v_k)}{\sum_{k=1}^K v_k}.$$

Exemple $v = (1, 0, 1)$: $P@1 = 1$, $P@2 = 0,5$, $P@3 = 0,667$.

$$\text{CP} = \frac{1 \cdot 1 + 0,5 \cdot 0 + 0,667 \cdot 1}{2} = \frac{1,667}{2} \approx 0,833.$$

(Récompense les pertinents placés *tôt*.)

24.4 Context Recall — arête $q \leftrightarrow C$ (avec référence)

On décompose la référence g en $\{g_1, \dots, g_m\}$, avec attribution $a_j \in \{0, 1\}$ au contexte :

$$\text{ContextRecall} = \frac{1}{m} \sum_{j=1}^m a_j.$$

Idée principale

Precision : le contexte est-il *propre* ? **Recall** : est-il *complet* ? Un recall bas = chunking/retrieval défaillant (l'info n'a pas été récupérée).

24.5 Answer Correctness (avec référence)

Combine un terme factuel (F1 sur affirmations TP/FP/FN) et un terme sémantique :

$$\text{F1} = \frac{|TP|}{|TP| + \frac{1}{2}(|FP| + |FN|)}, \quad \text{AC} = w_1 \text{F1} + w_2 \cos(E(a), E(g)), \quad (w_1 = 0,75, w_2 = 0,25).$$

Exemple $|TP| = 2$, $|FP| = 1$, $|FN| = 1$, $\text{sim} = 0,9$: $\text{F1} = \frac{2}{2+1} = 0,667$, $\text{AC} = 0,75 \cdot 0,667 + 0,25 \cdot 0,9 = 0,725$.

24.6 Autres & limites

Semantic Similarity = $\cos(E(a), E(g))$; Context Relevancy = $\frac{\text{\#phrases pertinentes}}{\text{\#phrases}}$; Noise sensitivity, Aspect Critique.

Pourquoi ça aide

RAGAS dépend d'un LLM-juge $\Rightarrow$ biais et variance. Bonnes pratiques : **temperature=0**, jeu de test suffisant, juge fort. RAGAS évolue : vérifier la version.

24.7 Code : RAGAS de bout en bout

```
1 from datasets import Dataset
2 from ragas import evaluate
3 from ragas.metrics import (faithfulness, answer_relevancy,
4                             context_precision, context_recall, answer_correctness)
5
6 samples = {"question": [], "answer": [], "contexts": [], "ground_truth": []}
7 for q, gt in zip(questions, references):
8     ctx = [d.page_content for d in retriever.invoke(q)]
9     ans = chain.invoke(q)
10    samples["question"].append(q);    samples["answer"].append(ans)
11    samples["contexts"].append(ctx); samples["ground_truth"].append(gt)
12
13 res = evaluate(Dataset.from_dict(samples), metrics=[
14     faithfulness, answer_relevancy,
15     context_precision, context_recall, answer_correctness])
16 print(res.to_pandas()) # un score par metrique et par question
```

25 Tableau de diagnostic

Symptôme	Diagnostic	Levier
Context recall bas	info non récupérée	chunking, embeddings, k , hybride
Context precision bas	contexte bruité/mal classé	reranking, MMR, filtrage
Faithfulness bas (ctx ok)	le LLM hallucine	prompt, garde-fous, LLM
Answer relevancy bas	réponse hors-sujet/vague	prompt, transform. requête
Answer correctness bas	faux ou incomplet	combinaison des leviers

Idée principale

On lit les métriques *ensemble*. Context recall haut mais faithfulness bas $\Rightarrow$ bon contexte mal exploité $\Rightarrow$ agir sur la génération. C'est ce qui rend l'optimisation ciblée.

26 Méthodologie : la boucle fermée

Idée principale

Baseline (mesurer toutes les métriques) $\rightarrow$ **identifier le maillon faible** (tableau de diagnostic) $\rightarrow$ **changer un seul levier** $\rightarrow$ **re-mesurer** $\rightarrow$ garder si gain. C'est ce qui transforme l'optimisation en démarche scientifique : chaque amélioration est *prouvée*.

Partie IV — Techniques RAG avancées

27 Self-RAG et Corrective RAG

Self-RAG (Asai et al., 2023) entraîne le LLM à émettre des tokens spéciaux [Retrieve], [No Retrieve], [Relevant], [Supported], etc., décidant *dynamiquement* s'il faut récupérer, et évaluant sa propre réponse.

Corrective RAG (CRAG, Yan et al., 2024) : un évaluateur léger classe chaque document en *correct*, *ambigu* ou *incorrect*. Si tous sont incorrects, on replie sur une recherche web. Si ambigu, on mixe.

28 Parent-Document Retrieval

On indexe de *petits* chunks (précision de la recherche) mais on renvoie au LLM le *document parent* (ou un chunk plus large) qui les contient (contexte complet). On découple ainsi l'unité de *recherche* de l'unité de *lecture*.

Pourquoi ça aide

Petits chunks $\Rightarrow$ embeddings spécifiques $\Rightarrow$ $\uparrow$ context precision ; parent large à la lecture $\Rightarrow$ le LLM dispose du contexte complet $\Rightarrow$ $\uparrow$ faithfulness.

29 GraphRAG

Au lieu (ou en plus) de chunker du texte plat, on extrait un **graphe de connaissance** : entités (nœuds) et relations (arêtes). La recherche peut alors exploiter la structure : « tous les voisins de l'entité X à distance 2 dont le type est Loi ». GraphRAG de Microsoft (2024) va plus loin en générant des résumés hiérarchiques de communautés du graphe, excellents pour des questions globales (« de quoi parle globalement ce corpus ? »).

30 Agentic RAG

Le retrieval n'est plus une étape unique mais un **outil** qu'un agent LLM peut appeler plusieurs fois, éventuellement avec des requêtes différentes selon l'information déjà obtenue. Cela permet le raisonnement **multi-sauts** : « trouve A , puis à partir de A cherche B , puis combine ».

Récapitulatif et conclusion

31 La chaîne de décisions

Étape	Décisions clés
Ingestion	Formats supportés, extraction de tables, OCR si scannés
Chunking	Taille T , recouvrement o , récursif vs sémantique vs late
Embedding	Symétrique ou asymétrique, dimension, préfixe, L_2 -norm
Index vectoriel	HNSW / IVF / PQ, métrique, paramètres ANN
Recherche	Dense seul, BM25 seul, hybride (linéaire ou RRF)
Transformation de requête	HyDE, multi-query, décomposition
Reranking	Cross-encoder sur top- K , modèle multilingue ou non
Prompt	Ordre des chunks, instruction de citation, template
Génération	Modèle, température, grounded vs libre
Évaluation	Jeu de test, métriques retrieval + génération

32 Synthèse générale

- **Pipeline.** Indexation (load $\rightarrow$ split $\rightarrow$ embed $\rightarrow$ store) puis récupération + génération ; le LLM est conditionné sur le contexte, formellement par marginalisation sur les k documents (RAG-

Sequence / RAG-Token).

- **Géométrie.** Similarité cosinus, produit scalaire et distance euclidienne sont *équivalents sur la sphère unité* — normalisez vos vecteurs.
- **Embeddings.** Le choix symétrique/asymétrique n'est pas cosmétique : ça change le modèle *et* l'usage (préfixes obligatoires sur l'asymétrie). La perte InfoNCE avec hard negatives est le cœur de tout bon encodeur.
- **Recherche.** L'architecture standard : hybride (BM25 + dense) fusionné par RRF, puis reranké par un cross-encoder.
- **Évaluation.** Triangle (q, C, a) : chaque métrique cible une arête. Familles : récupération (P/R@k, MRR, MAP, NDCG), génération classique (BLEU, ROUGE, BERTScore), LLM-juge, et RAGAS (faithfulness, answer relevancy, context precision/recall, answer correctness).
- **Optimisation.** chunking, RRF, MMR, reranking, HyDE/multi-query, compression, prompt, CRAG — chacun relié à une métrique, dans une **boucle fermée** baseline → levier → re-mesure. Sans évaluation, pas d'optimisation : instrumentez RAGAS *avant* de toucher aux hyperparamètres.

Pour aller plus loin. Lewis et al. 2020 (RAG original) ; Karpukhin et al. 2020 (DPR) ; Khattab & Zaharia 2020 (ColBERT) ; Gao et al. 2022 (HyDE) ; Asai et al. 2023 (Self-RAG) ; Yan et al. 2024 (CRAG) ; Edge et al. 2024 (GraphRAG) ; les familles de modèles BGE (BAAI) et E5 (Microsoft) pour les embeddings asymétriques open source.

Un RAG bien conçu n'est pas une bibliothèque plug-and-play : c'est un pipeline à multiples étapes, chacun avec ses hyperparamètres, et une boucle d'évaluation pour les calibrer.